

Progress Report 1

From publication archive to source-grounded research advisor

Prepared for supervisor review and thesis progress evidence

Generated July 3, 2026

134

Papers discovered/imported

SQLite local demo

98

Extracted PDFs

Full text locally available

756

Searchable chunks

Page-aware source evidence

39

Topics

Deterministic explorer index

Executive Summary

This first progress report documents the current state of the TTLAB Research Intelligence Platform, a local/demo research advisor that extends the original publication archive concept into full-paper discovery, source-grounded search, research question answering, thesis extension recommendations, paper intelligence artifacts, podcast script drafts, topic/author exploration, admin review, evaluation, and local LLM experimentation.

The platform has moved beyond a publication list. It now ingests TTLAB paper metadata and PDFs, extracts full-paper text, chunks papers into citable evidence units, indexes those chunks, retrieves relevant sources, generates structured AI-assisted outputs, and keeps every generated answer tied to citations, page ranges, review status, and warnings.

Current verification: backend tests pass with 66 tests; frontend TypeScript/Vite build passes; live API diagnostics report 134 papers, 98 extracted PDFs, 756 source chunks, 39 topics, 126 authors, 9 stored Ask answers, 16 thesis recommendations, 14 paper artifacts, and 9 local Ollama models available.

Important qualification: the system is demo-ready but not publication-ready. Generated outputs remain AI-assisted drafts. Admin review, citation checks, support statuses, and evaluation scaffolds are present so the project can be defended responsibly without overclaiming.

Table Of Contents

1. Project purpose and research framing
2. Current implementation snapshot
3. Architecture and data flow
4. Feature-by-feature progress
5. RAG, grounding, and local LLM baseline
6. Thesis Extension Finder and Paper Intelligence
7. Topic/Author Explorer, Admin Review, and Evaluation
8. Frontend and demo experience
9. Verification evidence
10. Current limitations, risks, and next work

1. Project Purpose And Research Framing

The project is titled TTLAB Research Intelligence Platform, with the working thesis framing: From Publication Archive to Research Advisor: A RAG-Based Platform for Academic Discovery. It extends the original paper, Automating the Collection, Display, Summarization and Podcasting of Academic Research, by shifting from publication collection and display toward full-paper understanding and student/researcher assistance.

The main research problem is that a publication archive is useful but shallow. Students and public users still need to know what papers are about, which papers match a question, which researchers appear connected to a topic, and which papers could become feasible thesis or project extensions. The platform addresses this by combining deterministic ingestion, full-paper extraction, retrieval, grounded answer generation, structured recommendation, review, and evaluation.

The system is intentionally scoped as a local/demo MVP. It does not attempt production deployment, authentication, production email, audio generation, or a complex graph database. It prioritizes a polished, explainable, source-grounded research intelligence workflow that can be demonstrated and defended within the master project timeline.

How This Extends The Original Publication Archive Work

Original archive direction	Current platform extension
Collect and display publications	Discover/import TTLAB metadata, download direct PDFs, and store reviewable records.
Summaries based mostly on abstracts/metadata	Generate paper intelligence from full-paper chunks with citations and support statuses.
Manual podcast preparation	Generate text-only podcast scripts with cited source chunks; no TTS/audio in the MVP.
Future RAG query support	Implemented Ask TTLAB with keyword/semantic/hybrid retrieval, citations, local Ollama answer composition, and grounding checks.
Public visibility of research	Add paper browser, search, Ask, thesis extension finder, paper intelligence, topic/author explorer, and related papers.
Manual quality control implied	Add Admin Review, review events, evaluation dashboard, and human review templates.

2. Current Implementation Snapshot

134 Total papers TTLAB records in SQLite	100 With PDF URL Direct PDF discovered	98 Downloaded PDFs Local demo artifacts	98 Extracted PDFs Text extraction complete
35 Missing PDFs Need review or no direct PDF	756 Chunks Indexed evidence units	98 Searchable papers At least one chunk	25 Semantic indexed chunks Hashing vector index
39 Topics Explorer topics	126 Authors Imported/aggregated	767 Paper-topic links Deterministic evidence links	1105 Author-topic links Derived from papers
9 Ask answers Stored Q&A runs	16 Extension ideas Generated recommendation items	14 Paper artifacts Summaries/scripts/etc.	5 Podcast scripts Text-only scripts

The current local dataset is larger than the initial few-paper MVP. It contains 134 papers and 756 full-paper chunks. This is useful for demonstrating breadth, but it also creates a review backlog: 134 papers, 9 Ask answers, 4 recommendation runs, and 14 artifacts currently need review. That backlog is expected because the Admin Review workflow exists but no reviewer approvals have been recorded yet.

Top Explorer Topics

Topic	Papers	Authors	Meaning In Demo
Data Science	67	81	Deterministically linked to 67 indexed TTLAB paper(s).
Networks	67	74	Deterministically linked to 67 indexed TTLAB paper(s).
Education	66	79	Deterministically linked to 66 indexed TTLAB paper(s).
Machine Learning	53	69	Deterministically linked to 53 indexed TTLAB paper(s).
Visualization	44	63	Deterministically linked to 44 indexed TTLAB paper(s).
Web Applications	38	46	Deterministically linked to 38 indexed TTLAB paper(s).

3. Architecture And Data Flow

The backend is a FastAPI application using SQLAlchemy and SQLite. The frontend is React, Vite, and TypeScript. Local data lives under data/ and generated runtime artifacts such as PDFs, extracted text, chunks, indexes, generated outputs, and SQLite DB files are ignored by Git. This preserves reproducibility without committing bulky generated artifacts.

Stage	Current behavior
1. Discover	Scrape TTLAB WordPress publication archive, preserve raw metadata, classify PDF/source links.
2. Import	Upsert papers/authors into SQLite with review_status=needs_review.
3. Download	Safely download direct PDFs only, dry-run by default, deterministic filenames.
4. Extract	Use PyMuPDF for page-aware text extraction and scanned/weak-text diagnostics.
5. Chunk	Create deterministic page/section-aware chunks with source hashes.
6. Index	Build keyword FTS and local semantic hashing index for offline search.
7. Retrieve	Run keyword, semantic, or hybrid retrieval with expansion, metadata boosts, section boosts, and diversity.
8. Generate	Use offline deterministic or local Ollama providers to draft answers/artifacts with citations.
9. Verify	Check citations, grounding, support status, warnings, and review state.
10. Review	Use Admin Review and Evaluation Dashboard to avoid overclaiming generated outputs.

Backend Subsystems

Subsystem	Responsibility	Key modules
API	Expose papers, stats, search, Ask, recommendations, artifacts, admin review, evaluation, explorer, and local LLM endpoints.	backend/app/api/*
Models	Persist papers, authors, chunks, RAG answers, thesis recommendations, artifacts, review events, topics, and relationships.	backend/app/models/*
Ingestion	Discover TTLAB publications, import seed data, safely download PDFs, extract text.	ingestion/ttlab_page.py, manual_import.py, pdf_downloader.py, pdf_parser.py
Indexing	Chunk extracted text, build keyword and semantic indexes, retrieve evidence.	chunker.py, keyword_search.py, embedder.py, vector_store.py, retriever.py
Intelligence	Create grounded answers, recommendations, paper artifacts, podcast scripts, topic/author links, and verifiers.	rag_answerer.py, extension_recommender.py, paper_artifact_generator.py, topic_explorer.py
Evaluation	Run/read retrieval, QA, extension, artifact, and local LLM benchmark results.	evaluation/*.py

4. Feature-By-Feature Progress

Feature area	What is done	Status
Ingestion	TTLAB archive scraper, seed JSON, manual import, direct-PDF downloader	Working
Full-paper processing	PyMuPDF extraction, page diagnostics, deterministic chunks	Working
Search/Retrieval	Keyword FTS, hashing semantic search, hybrid retrieval with smarter ranking	Working
Ask TTLAB	Citation-grounded question answering, local Ollama selector, paper-specific mode	Working
Thesis Extension Finder	Structured cited recommendations, fit rationale, MVP/evaluation plans	Working
Paper Intelligence	Public/technical summaries, methods, limitations, future work, skills, evaluation plans	Working
Podcast scripts	Text-only dialogue scripts with citations	Working
Explorer	Topics, authors, expertise summaries, related papers	Working
Admin Review	Review queue, status updates, notes, audit events	Working
Evaluation	Retrieval/QA/extension/artifact result dashboard and templates	Working

Publication Discovery And Import

The scraper starts from the TTLAB publication archive, follows controlled pagination, parses WordPress publication entries, resolves relative URLs, classifies direct PDFs versus source/DOI/publisher links, preserves all discovered URLs, and outputs seed JSON. The importer upserts papers and authors into SQLite while keeping uncertain metadata under review. No metadata is silently invented.

PDF Download, Extraction, And Chunking

The downloader only saves direct PDF URLs and defaults to dry-run unless explicitly told to download. It verifies content type or PDF signature and stores deterministic filenames. PyMuPDF extraction records page counts, text counts, weak/scanned PDF warnings, and extraction errors. The chunker creates deterministic page-aware chunks with chunk IDs, page ranges, sections, text, token estimates, and source hashes.

Search And Retrieval

Search supports keyword, semantic, and hybrid modes. Keyword uses SQLite FTS5 where available, with deterministic fallback matching. Semantic search uses local hashing embeddings, which keep the system offline and simple. Hybrid retrieval now adds deterministic query expansion, metadata-aware boosts, section-aware boosts, and diversity ranking so short queries such as RAG, AI, ML, and optimization have better odds of retrieving the right evidence.

5. RAG, Grounding, And Local LLM Baseline

Ask TTLAB is not a general chatbot. It retrieves source chunks first, passes compact cited evidence to an answer provider, builds citations from retrieved chunks, verifies citation grounding, stores the answer, and exposes review metadata. Answers are marked grounded, partial, or unsupported based on citations and support checks.

Ask TTLAB Current Diagnostics

9	9	0	0
Stored answers	Grounded answers	Partial answers	Unsupported answers
Persisted RAG answers	Citation-supported	Weak or inferred support	No usable support

Local Ollama Integration

Local Ollama models have been integrated as optional answer composers. The default API behavior remains safe and offline unless Ollama is explicitly selected or configured. The frontend Ask page now includes a model selector, provider selector, paper-scope selector, traffic-light model quality badge, benchmark metadata area, and warnings when Ollama is unavailable.

Model tier	Models	Current interpretation
Green	qwen3:4b-instruct-2507-q4_K_M; gemma3:4b-it-q4_K_M	Recommended first for local Ask TTLAB on the current hardware.
Light green	phi3.5:3.8b-mini-instruct-q4_K_M; ministral-3:3b-instruct-2512-q4_K_M; llama3.2:3b	Good practical baselines.
Yellow	phi3:mini; smollm2:1.7b	Usable but quality should be reviewed carefully.
Red	smollm2:360m; qwen2.5:0.5b	Not recommended except for speed comparisons.

Current local LLM status: Ollama available=True; model_count=9; default_model=qwen3:4b-instruct-2507-q4_K_M. Latest persisted full benchmark status=not_run. A one-model live smoke benchmark was run separately during implementation, but the full installed-model benchmark is intentionally left for a longer local GPU run.

Why RAG Is Now Smarter

- Query expansion maps abbreviations and spelling variants to richer terms, for example RAG to retrieval augmented generation, AI to artificial intelligence, ML to machine learning, and optimization to optimisation.
- Metadata scoring gives a small boost when query terms appear in paper titles, authors, venues, or topics.
- Section boosts prioritize methods, results, limitations, future work, abstracts, introductions, or conclusions depending on the question wording.
- Diversity ranking reduces repeated adjacent chunks from the same paper so the answer has better source coverage.
- Paper-specific Ask mode allows a student to ask only about one selected paper from Paper Detail.

6. Thesis Extension Finder And Paper Intelligence

Thesis Extension Finder

The Thesis Extension Finder is the most differentiated student-facing feature. It collects interests, skills, timeline, project type, data constraints, difficulty, preferred topics, avoid topics, retrieval mode, and top-k. It converts the student profile into retrieval queries, retrieves chunks, groups by paper, scores candidates, and generates structured recommendations.

Recommendation field	Purpose
Paper focus	Separates what the source paper actually says or does.
Source-supported facts	Claims mapped to retrieved chunks with snippets and page ranges.
Identified gap	Marks whether limitations/future work are explicit, inferred, or not found.
Extension title and summary	Clearly labeled system suggestion, not verified paper fact.
Why it fits student	Connects paper, interests, skills, timeline, project type, and data constraints.
MVP, stretch goals, skills gap, data availability, difficulty, risk, implementation time	Makes the recommendation actionable for a thesis or semester project.
Evaluation plan	Explains how the student could measure success.
Potential researcher fit	Based on authorship only, never a confirmed supervisor claim.

Current recommendation diagnostics: 4 runs, 16 recommendations, grounding split grounded=0, partial=4, unsupported=0.

Paper Intelligence And Podcast Scripts

Paper Intelligence generates structured paper-level outputs from full-paper chunks: public summary, technical summary, contribution, methods, limitations, future work, possible extensions, required skills, evaluation plan, and a 3-5 minute text-only podcast script. Every artifact stores citations where possible, source chunk IDs, provider/model metadata, grounding status, warnings, and review status.

5	14	9	5
Papers with artifacts	Total artifacts	Grounded artifacts	Partial artifacts
Generated locally	All artifact rows	Citation-supported	Some inferred/weak support

7. Topic/Author Explorer, Admin Review, And Evaluation

Topic/Author Explorer

The explorer normalizes deterministic topics from paper metadata, titles, venues, chunks, and generated artifacts. It creates PaperTopic and AuthorTopic links with evidence JSON so the UI can explain why a topic was assigned. Author profiles summarize papers, top topics, coauthors, venues, generated artifacts count, and source-derived expertise. The wording avoids claiming that an author is a supervisor or verified expert beyond indexed authorship evidence.

39 Topics Explorer index	126 Authors Author profiles	767 Paper-topic links Evidence-backed	1105 Author-topic links Derived from papers
---------------------------------------	--	--	--

Admin Review

The Admin Review page is a local/demo workflow with no authentication. It supports review queue filtering, metadata correction, artifact review, Ask answer review, thesis recommendation review, extraction review, and review event audit trails. This makes the platform academically responsible by making generated or extracted content reviewable instead of treating it as automatically correct.

Review area	Current count needing review
Paper metadata	134
Ask answers	9
Thesis recommendation runs	4
Paper artifacts	14
Review events recorded	0

Evaluation Dashboard

The Evaluation Dashboard reads result JSON files without inventing missing metrics. Missing retrieval and QA result files appear as not_run. Extension and artifact evaluation result files are present, but they are still only as meaningful as the reviewed cases behind them. Human review templates exist for deeper thesis evaluation.

Evaluation area	Status	Key current result
Retrieval	not_run	No result file present yet.
QA	not_run	No result file present yet.
Extension	available	cases=1; citation_coverage=1.0
Artifact	available	cases=1; citation_coverage=1.0

8. Frontend And Demo Experience

The frontend is organized as a public/demo research intelligence interface rather than a landing page. It uses dashboard metrics, paper browsing, paper detail, search, Ask TTLAB, Thesis Extension Finder, Topic/Author Explorer, Admin Review, and Evaluation views. Recent UI polish added loading skeletons, empty states, progress states, status badges, toast notifications, consistent cards/tables, and clearer warning language.

Route/page	What the user can do
Dashboard	View papers, PDFs, chunks, topics, artifacts, review/evaluation counts, and recent papers.
Papers	Browse/filter TTLAB paper metadata and open paper detail.
Paper Detail	Inspect metadata, extraction status, chunks, related papers, paper intelligence, podcast script, and ask about one paper.
Search	Run keyword, semantic, or hybrid search over full-paper chunks with snippets and citations.
Ask TTLAB	Ask citation-grounded questions, choose local/offline provider, choose Ollama model, and inspect retrieved chunks.
Thesis Extension Finder	Enter student profile and get structured, cited project recommendations.
Topic/Author Explorer	Browse topics, authors, expertise summaries, papers, and related work.
Admin Review	Review/correct items and track audit events.
Evaluation	Inspect available/not_run evaluation result files without invented metrics.

Demo launcher: scripts/run_everything.sh prepares bounded data, verifies backend/frontend, runs smoke checks, and starts dev servers. Useful variants include --serve-only, --verify-only, --prepare-only, --skip-downloads, and --skip-artifacts.

9. Verification Evidence

The latest full verification completed successfully after the local Ollama defaults were adjusted so tests remain offline by default. Explicit Ollama behavior is covered by mocked tests and a one-model live smoke benchmark was also completed during implementation.

Verification item	Latest observed result
Backend test suite	PYTHONPATH=backend .venv/bin/python -m pytest -> 66 passed, 5 warnings.
Frontend build	cd frontend && npm run build -> TypeScript/Vite build passed.
API smoke checks	/api/lms/local, /api/lms/benchmark/latest, /api/ask/diagnostics returned 200.
Ollama availability	ollama list returned 9 installed local models.
Quick local benchmark	qwen2.5:0.5b completed 3 cases cold/warm with avg 3.535 seconds and 379.29 tok/s; full benchmark intentionally not run to completion in this turn.
PDF generation dependencies	ReportLab, pdfplumber, pypdf installed in .venv; Poppler pdftoppm available for render checks.

Quality Controls Built Into The Product

- Generated content is labeled AI-assisted and unreviewed until approved.
- Source-supported facts, future-work claims, and system suggestions are separated in recommendation and artifact outputs.
- Citations include paper ID/title, chunk ID, section, page range, snippet, and score where applicable.
- Grounding statuses are explicit: grounded, partial, unsupported.
- Review statuses are explicit: needs_review, reviewed, approved, rejected, needs_reprocess.
- Evaluation dashboards show not_run instead of inventing metrics for missing files.
- Local data artifacts are ignored so the repository stays reproducible and lightweight.

10. Current Limitations, Risks, And Next Work

Current Limitations

- No authentication or role-based access control is implemented. Admin Review is a local/demo tool.
- No production deployment infrastructure is included.
- No production email notifications are implemented.
- No audio/TTS generation is implemented; podcast output is text-only.
- No OCR is implemented for scanned or image-heavy PDFs.
- Semantic search currently uses local hashing embeddings by default; BGE-M3 or another local embedding model would likely improve retrieval quality.
- Author identity normalization is basic, so variants such as Patrick Hosein and P. Hosein may remain separate unless reviewed.
- Topic labels and author expertise summaries are deterministic/inferred unless reviewed.
- Local LLM model colors are preliminary until the full benchmark is run and manually reviewed.
- Evaluation metrics should not be claimed as research quality until gold files and human scores are manually reviewed.

Recommended Next Work

Priority	Next action	Why it matters
High	Run the full Ollama benchmark across installed models and save results under data/evaluation.	Turns preliminary model colors into local evidence for the supervisor/demo.
High	Add manually reviewed retrieval and QA gold files.	Makes evaluation claims defensible.
High	Review/approve a representative set of paper metadata, Ask answers, recommendations, and artifacts.	Shows the admin workflow is not only present but usable.
Medium	Pull and test bge-m3 for local embeddings.	Likely improves retrieval quality more than simply using larger answer models.
Medium	Normalize common author aliases after review.	Improves author explorer credibility.
Medium	Capture screenshots for the supervisor demo deck.	Makes the project easier to present.

Overall progress: the platform is now a cohesive local research intelligence system with ingestion, full-paper indexing, search, grounded question answering, local LLM support, thesis advising, generated paper intelligence, topic/author exploration, admin review, evaluation scaffolding, and demo tooling. The largest remaining work is not feature volume; it is review, evaluation evidence, and polishing the data/model quality story.

Appendix A: Command Reference

Goal	Command
Run all backend tests	<code>PYTHONPATH=backend .venv/bin/python -m pytest</code>
Build frontend	<code>cd frontend && npm run build</code>
Start backend	<code>uvicorn app.main:app --reload --app-dir backend</code>
Start frontend	<code>cd frontend && npm run dev</code>
One-command demo	<code>./scripts/run_everything.sh</code>
Serve only	<code>./scripts/run_everything.sh --serve-only</code>
Prepare data	<code>PYTHONPATH=backend .venv/bin/python -m app.demo.prepare_demo --limit 25</code>
Rebuild keyword index	<code>PYTHONPATH=backend .venv/bin/python -m app.indexing.keyword_search rebuild</code>
Rebuild semantic index	<code>PYTHONPATH=backend .venv/bin/python -m app.indexing.embedder index --provider hashing</code>
Rebuild topics	<code>PYTHONPATH=backend .venv/bin/python -m app.intelligence.topic_explorer rebuild</code>
Run Ollama benchmark	<code>PYTHONPATH=backend .venv/bin/python -m app.evaluation.ollama_benchmark --models installed</code>

Appendix B: API Highlights

Endpoint	Purpose
<code>/health</code>	Backend health.
<code>/api/stats</code>	Dashboard-wide counts and status.
<code>/api/papers</code>	Paper browser data.
<code>/api/search?q=RAG&mode=hybrid</code>	Chunk-level retrieval.
<code>/api/ask</code>	Citation-grounded question answering with optional <code>paper_id</code> and local model fields.
<code>/api/lms/local</code>	Local Ollama model list, colors, recommendations, benchmark summary.
<code>/api/recommendations/extensions</code>	Thesis Extension Finder.
<code>/api/papers/{paper_id}/artifacts/generate</code>	Paper intelligence and text podcast script generation.
<code>/api/explorer/overview</code>	Topic/Author Explorer dashboard.
<code>/api/admin/overview</code>	Local review status and audit overview.
<code>/api/evaluation/dashboard</code>	Read-only evaluation summary.